EXAMEN RDA-2 DE PROGRAMACIÓN MÓVIL

1. Título del Examen

EcoTrip 2026: Sistema Resiliente de Planificación de Rutas e Identidad Visual Adaptativa

2. Objetivos del Examen

1. **Arquitectura de Flujo Segura:** Orquestar un sistema de navegación progresiva y fuertemente tipada (*Type-Safety*) utilizando rutas basadas en clases @Serializable, asegurando la correcta manipulación de argumentos y la limpieza de la pila de pantallas (*Back Stack*).
2. **Resiliencia Multinivel de Datos:** Implementar un mecanismo de persistencia local que conserve las preferencias del usuario en disco mediante DataStore y mitigue la pérdida de información en formularios reactivos frente a cambios de configuración o muerte del proceso (*Process Death*) mediante ViewModel y SavedStateHandle.
3. **Estética Semántica y Adaptativa:** Diseñar interfaces gráficas de usuario completamente declarativas en *Jetpack Compose* alineadas con las directrices de *Material Design 3*, forzando la integración del Color Dinámico (*Material You*) y el uso de componentes estructurales avanzados.

3. Contexto

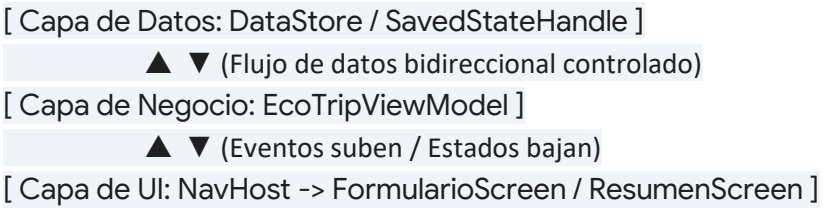
En el contexto global de la movilidad sostenible de 2026, la empresa de turismo ecológico *EcoTrip* requiere una aplicación móvil nativa para que sus usuarios planifiquen y configuren sus perfiles de viaje. El núcleo del problema radica en la volatilidad de la experiencia del usuario en dispositivos móviles modernos: si un usuario está configurando su ruta y decide rotar la pantalla para tomar una fotografía o si el sistema operativo destruye temporalmente la aplicación en segundo plano por falta de memoria, los datos introducidos no pueden perderse en ninguna circunstancia.

Los estudiantes deberán desarrollar una solución compuesta por dos pantallas principales, donde la primera actúe como un formulario ultra-resiliente y de configuración global, y la segunda sirva como un panel dinámico de confirmación de ruta que reciba objetos de datos complejos de forma segura y cuya interfaz se adapte visualmente a las elecciones estéticas del sistema operativo del usuario.

4. Arquitectura General de la Aplicación

Para garantizar el cumplimiento de los estándares de ingeniería de software revisados en el

curso, la aplicación debe estructurarse sin lógica acoplada, separando limpiamente la capa de presentación de la capa de datos bajo el patrón de diseño de Flujo Unidireccional de Datos (UDF).



Componentes de Datos y Modelado:

- **Modelo de Datos Inmutable:** Se creará una data class llamada PreferenciaViaje con propiedades inmutables (val) para representar las opciones fijas del usuario: nombre del viajero, tipo de transporte ecológico (tren, bicicleta, vehículo eléctrico) y un interruptor que indique si prefiere exclusivamente rutas de baja huella de carbono.
- **Rutas Tipadas:** Las pantallas se definirán fuera del grafo de navegación como objetos o clases serializables utilizando la anotación @Serializable de Kotlin. La ruta hacia la pantalla de resumen debe exigir el paso obligatorio de un objeto complejo serializable o, en su defecto, parámetros estrictos (por ejemplo: val destino: String, val diasDuracion: Int, val esViajeGrupal: Boolean) que el compilador verifique en tiempo de compilación.

Gestión de Estados y Persistencia:

- **Persistencia en Disco (DataStore):** Se creará una clase de gestión encargada de almacenar de forma asíncrona las preferencias globales (ej. el nombre del usuario y el interruptor de huella de carbono) para que persistan de manera permanente, incluso si la aplicación se cierra por completo.
- **Persistencia de Proceso (SavedStateHandle):** Dentro del EcoTripViewModel, se enlazará el estado del destino del viaje y la duración temporal al contenedor de estado de Jetpack para blindar los campos del formulario frente a una destrucción forzada por el sistema (*Process Death*).
- **ViewModel Unificado:** Actuará como la única fuente de verdad (*Single Source of Truth*). Expondrá un estado de interfaz reactivo que la UI consumirá en tiempo real.

Capa de Interfaz de Usuario (UI Layer):

- **Estructura Base:** Cada pantalla deberá implementarse dentro de un contenedor Scaffold, aprovechando los parámetros semánticos nativos para organizar de manera limpia la barra superior (TopAppBar) y los botones de acción flotantes (FloatingActionButton).
- **Navegación:** Un único NavHost controlará el flujo. Se debe programar la lógica del botón

de retorno de manera que, al salir de la pantalla de resumen, se limpie explícitamente la pila utilizando popUpTo con la configuración adecuada para evitar la acumulación de instancias redundantes de la pantalla anterior, aplicando la bandera launchSingleTop.

5. Resultados Esperados

Al ejecutar la aplicación en un dispositivo virtual configurado bajo la **API 36 (Android 16)**, se deberán validar con precisión los siguientes comportamientos operativos:

1. **Comportamiento en Pantalla 1 (Configuración del Viaje):**
    - La interfaz debe presentar campos de entrada de texto legibles, componentes de selección para el medio de transporte y un interruptor para la huella de carbono.
    - **Prueba de Rotación:** Al escribir información en el campo de texto y rotar el emulador a modo horizontal, la interfaz debe redibujarse fluidamente (recomposición) sin borrar ni una sola letra del formulario.
    - **Prueba de Muerte de Proceso:** Con la opción *"Don't keep activities"* (No conservar actividades) activada en los ajustes de desarrollador del emulador, al presionar el botón de inicio de Android (mandando la app a segundo plano) y volver a abrirla, los datos temporales del formulario deben autorepararse y mostrarse intactos gracias a la persistencia multinivel.
2. **Transición y Comportamiento en Pantalla 2 (Resumen Tipado):**
    - Al hacer clic sobre el botón de acción principal de la pantalla 1, la aplicación debe validar las entradas (evitando valores nulos mediante operadores seguros como ?: y conversores como toIntOrNull()) y viajar hacia la pantalla de resumen inyectando los tipos definidos de forma segura.
    - La pantalla de resumen mostrará de forma formateada mediante plantillas de cadena idiomáticas de Kotlin toda la configuración del viaje.
    - **Prueba de Identidad Visual (Material You):** Al ingresar a los ajustes del sistema del emulador y cambiar el fondo de pantalla (*Wallpaper*), la paleta de colores del TopAppBar y de las superficies de la aplicación deben cambiar de tonalidad instantáneamente al regresar a la app, demostrando que dependen dinámicamente de MaterialTheme.colorScheme y no de valores hexadecimales estáticos (*hardcoded*).
    - El botón de salida integrado en la barra superior debe devolver al usuario a la pantalla inicial destruyendo el historial de transiciones previo en la pila.

6. Entregables

Cada grupo de trabajo deberá consolidar y cargar en la plataforma Moodle un único archivo comprimido que contenga rigurosamente los siguientes tres elementos:

1. **Proyecto en Android Studio (.zip):** La carpeta completa del código fuente de la

aplicación desarrollada, estructurada bajo las especificaciones técnicas estandarizadas en clase:
    - Compilado con **compileSdk = 36** y **targetSdk = 36**.
    - Construido sobre el entorno de ejecución **Java 21 / JBR 21**.
    - Uso de Jetpack Compose con dependencias blindadas en el catálogo de versiones (libs.versions.toml).
2. **Control de Versiones (Evidencia en GitHub):** El enlace (URL) público al repositorio de GitHub donde se desarrolló el examen. El historial de control de cambios debe reflejar un flujo de trabajo profesional con **al menos 4 commits descriptivos** firmados por los integrantes del grupo, evidenciando de manera progresiva hitos como: *"feat: data models and routes"*, *"feat: viewmodel and dataStore implementation"*, *"feat: compose ui layout with material design 3"*, y *"feat: type-safe navigation and testing"*.
3. **Documento de Informe Técnico (PDF):** Un reporte formal con nivel de ingeniería que incluya:
    - **Portada formal** con la identificación de los integrantes del grupo de trabajo.
    - **Descripción de la arquitectura** implementada (diagrama conceptual o explicación de cómo interactúan la data class, el ViewModel, SavedStateHandle y DataStore).
    - **Evidencias Gráficas (Capturas de Pantalla completas del Emulador API 36):**
        1. Captura del Formulario en ejecución con datos válidos ingresados.
        2. Captura del Formulario reflejando el éxito de la prueba de rotación o persistencia de estado.
        3. Captura de la Pantalla de Resumen evidenciando el acoplamiento cromático al Color Dinámico del sistema (Material You).
    - **Explicación Conceptual:** Una justificación técnica fundamentada sobre por qué el uso de la navegación fuertemente tipada (*Type-Safety*) incorporada en la arquitectura moderna de Android reduce los errores en tiempo de ejecución (*crashes*) en comparación con el antiguo paso de parámetros por cadenas de texto (*Strings*) o *Bundles* manuales.
    - **Conclusiones y Recomendaciones** de ingeniería basadas en los resultados obtenidos en las pruebas de resiliencia.
    - Bibliografía.
    - Copia de todo el código de la aplicación como Anexo.
    - Cumplimiento de las normas APA.

7. RÚBRICA DE EVALUACIÓN

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.